HomeMade Pickles & Snacks: Taste the Best

Project Description:

HomeMade Pickles & Snacks - Taste the Best is a cloud-based culinary platform revolutionizing access to authentic, handcrafted pickles and snacks. Addressing the growing demand for preservative-free, traditional recipes, this initiative combines artisanal craftsmanship with cutting-edge technology to deliver farm-fresh flavours directly to consumers. Built on Flask for backend efficiency and hosted on AWS EC2 for scalable performance, the platform offers seamless browsing, ordering, and subscription management. DynamoDB ensures real-time inventory tracking and personalized user experiences, while fostering sustainability through partnerships with local farmers and eco-friendly packaging. From tangy regional pickles to wholesome snacks, every product celebrates heritage recipes, nutritional integrity, and convenience - proving that tradition and innovation can coexist deliciously. "Preserving Traditions, One Jar at a Time."

Scenario 1: Scalable Order Management for High Demand

A cloud-based system ensures seamless order processing during peak user activity. For instance, during a promotional event, hundreds of users simultaneously access the platform to place orders. The backend efficiently processes requests, updates inventory in real-time, and manages user sessions. The cloud infrastructure handles traffic spikes without performance degradation, ensuring smooth transactions and minimizing wait times.

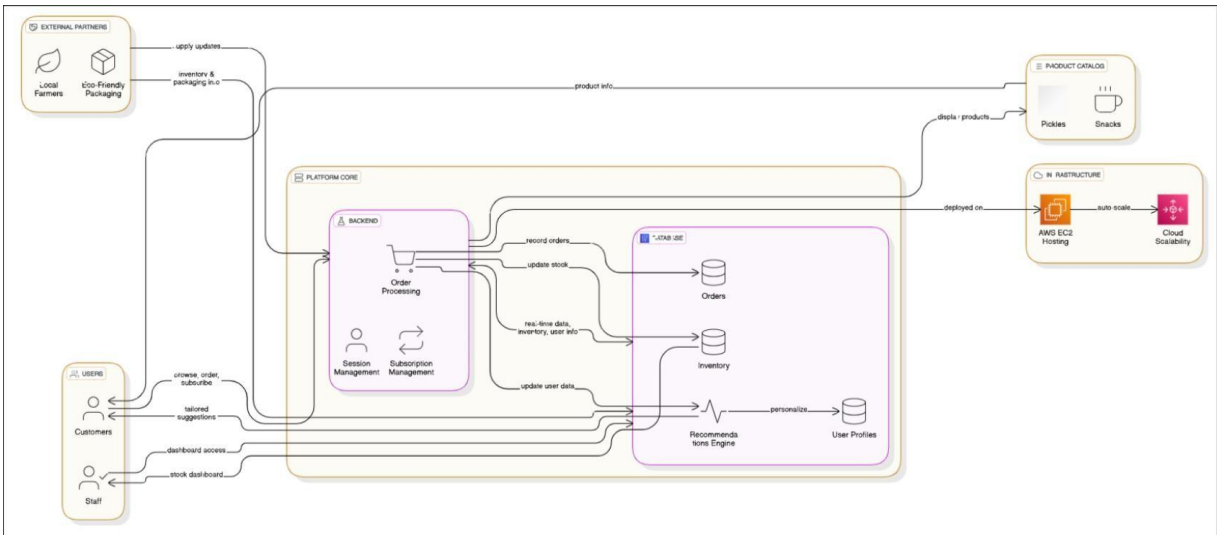
Scenario 2: Real-Time Inventory Tracking and Updates

When a customer places an order for a product, the system instantly updates stock levels and records transaction details. For example, a user purchases an item, triggering automatic inventory deduction and order confirmation. Staff members receive updated dashboards to monitor stock availability and fulfilment progress, ensuring timely restocking and minimizing overselling risks.

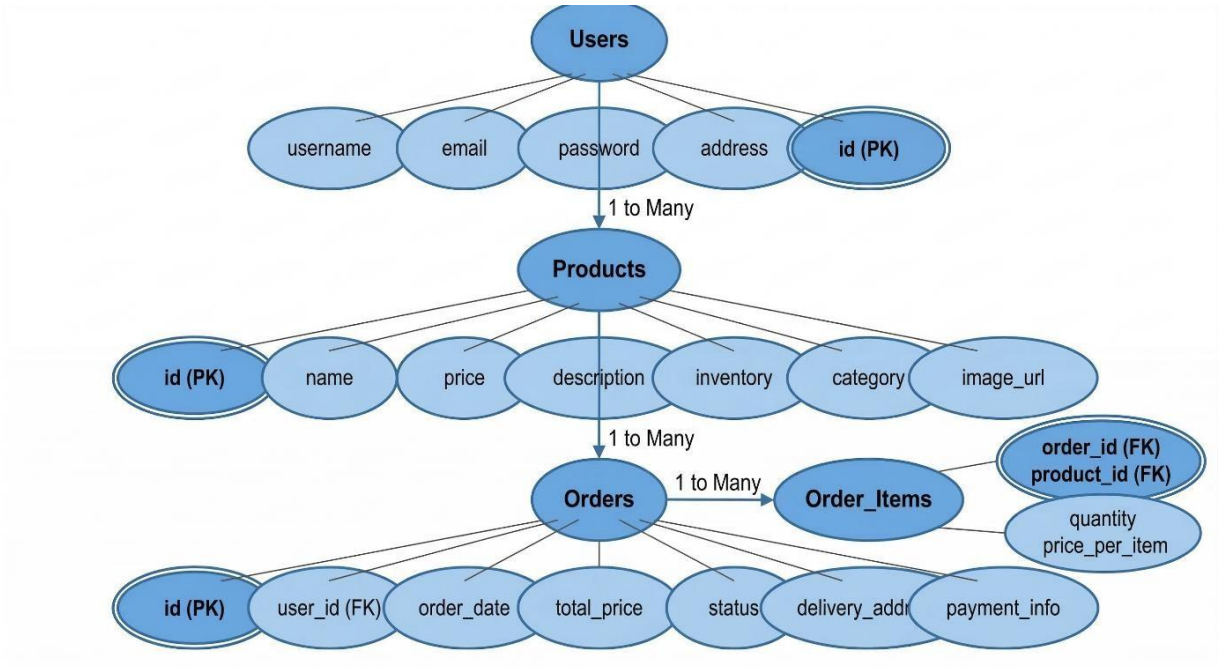
Scenario 3: Personalized User Experience and Recommendations

The platform leverages user behavior data to enhance engagement. A returning customer, for instance, views tailored recommendations based on past purchases and browsing history. The system dynamically adjusts suggestions in real-time, while maintaining fast response rates even during high traffic, creating a frictionless and intuitive shopping experience.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

1. **AWS Account Setup**: AWS Account Setup
2. **AWS IAM (Identity and Access Management)**: IAM Overview
3. **AWS EC2 (Elastic Compute Cloud)**: EC2 Guide
4. **AWS DynamoDB**: DynamoDB Introduction
5. **SNS Overview**: SNS Documentation
6. **Git Version Control**: Git Documentation

Project Workflow:

1. Backend Development and Application Setup

Activity 1.1: Develop the Backend Using Flask.

Activity 1.2: Integrate AWS Services Using boto3.

2. AWS Account Setup and Login

Activity 2.1: Set up an AWS account if not already done.

Activity 2.2: Log in to the AWS Management Console

3. DynamoDB Database Creation and Setup

Activity 3.1: Navigate to the DynamoDB.

Activity 3.2: Create a DynamoDB table for storing data

4. IAM Role Setup

Activity 4.1: Create IAM Role

Activity 4.2: Attach Policies

5. EC2 Instance Setup

Activity 5.1: Launch an EC2 instance to host the Flask application.

Activity 5.2: Configure security groups for HTTP, and SSH access.

6. Deployment on EC2

Activity 6.1: Upload Flask Files

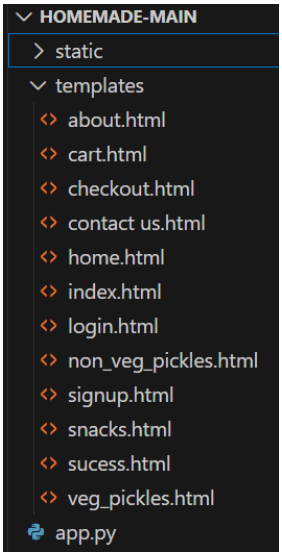
Activity 6.2: Run the Flask App

7. Testing and Deployment

Activity 7.1: Conduct functional testing to verify the Project

Milestone 1: Backend Development and Application Setup

- **Activity 4.1: Develop the backend using Flask**
  - File Explorer Structure:



Description: Create a templates folder containing HTML files for each major feature—home, about, product categories (veg pickles, non-veg pickles, snacks), user account (login, signup), and the checkout flow (cart, checkout, success)—and manage backend logic in app.py.

Description of the code:

- **Flask App Initialization**

  Import Libraries:

```
from flask import Flask, render_template, request, redirect, url_for, session
from werkzeug.security import generate_password_hash, check_password_hash
import boto3
from datetime import datetime
import json,uuid
```

Description: Import necessary Flask modules for web routing and session management, Werkzeug for secure password hashing, Boto3 for AWS service access, datetime for timestamps, and JSON and UUID for data handling and unique identifiers.

Initialize flask app:

```
app = Flask(__name__)
```

Description: Initialize the Flask application by creating an instance of the Flaskclass with the current module name.

DynamoDB Connection:

```
dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')  # e.g., 'us-east-1'
users_table = dynamodb.Table('Users')
orders_table = dynamodb.Table('Orders')
```

Description Use boto3 to connect to DynamoDB for handling user registration, Order details database operations and also mention region name where DynamoDB tables are created

```
products = {
    'non_veg_pickles': [
        {'id': 1, 'name': 'Chicken Pickle', 'weights': {'250': 600, '500': 1200, '1000': 1800}},
        {'id': 2, 'name': 'Fish Pickle', 'weights': {'250': 200, '500': 400, '1000': 800}},
        {'id': 3, 'name': 'Gongura Mutton', 'weights': {'250': 400, '500': 800, '1000': 1600}},
        {'id': 4, 'name': 'Mutton Pickle', 'weights': {'250': 400, '500': 800, '1000': 1600}},
        {'id': 5, 'name': 'Gongura Prawns', 'weights': {'250': 600, '500': 1200, '1000': 1800}},
        {'id': 6, 'name': 'Chicken Pickle (Gongura)', 'weights': {'250': 350, '500': 700, '1000': 1050}}
    ],
    'veg_pickles': [
        {'id': 7, 'name': 'Traditional Mango Pickle', 'weights': {'250': 150, '500': 280, '1000': 500}},
        {'id': 8, 'name': 'Zesty Lemon Pickle', 'weights': {'250': 120, '500': 220, '1000': 400}},
        {'id': 9, 'name': 'Tomato Pickle', 'weights': {'250': 130, '500': 240, '1000': 450}},
        {'id': 10, 'name': 'Kakarakaya Pickle', 'weights': {'250': 130, '500': 240, '1000': 450}},
        {'id': 11, 'name': 'Chintakaya Pickle', 'weights': {'250': 130, '500': 240, '1000': 450}},
        {'id': 12, 'name': 'Spicy Pandu Mirchi', 'weights': {'250': 130, '500': 240, '1000': 450}}
    ],  # Add your veg pickle products here
    'snacks': [
        {'id': 7, 'name': 'Banana Chips', 'weights': {'250': 300, '500': 600, '1000': 800}},
        {'id': 8, 'name': 'Crispy Aam-Papad', 'weights': {'250': 150, '500': 300, '1000': 600}},
        {'id': 9, 'name': 'Crispy Chekka Pakodi', 'weights': {'250': 50, '500': 100, '1000': 200}},
        {'id': 10, 'name': 'Boondhi Acchu', 'weights': {'250': 300, '500': 600, '1000': 900}},
        {'id': 11, 'name': 'Chekkalu', 'weights': {'250': 350, '500': 700, '1000': 1000}},
        {'id': 12, 'name': 'Ragi Laddu', 'weights': {'250': 350, '500': 700, '1000': 1000}},
        {'id': 13, 'name': 'Dry Fruit Laddu', 'weights': {'250': 500, '500': 1000, '1000': 1500}},
        {'id': 14, 'name': 'Kara Boondi', 'weights': {'250': 250, '500': 500, '1000': 750}},
        {'id': 15, 'name': 'Gavvalu', 'weights': {'250': 250, '500': 500, '1000': 750}},
        {'id': 16, 'name': 'Kaju Chikki', 'weights': {'250': 250, '500': 500, '1000': 750}},
```

- **Routes for Web Pages**

Register User:

```python
@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        username = request.form['username'].strip()
        email = request.form['email'].strip()
        password = request.form['password']

        try:
            # Check if username exists
            response = users_table.get_item(Key={'username': username})
            if 'Item' in response:
                return render_template('signup.html', error='Username already exists')

            # Hash password before storing
            hashed_password = generate_password_hash(password)

            # Store new user in DynamoDB
            users_table.put_item(
                Item={
                    'username': username,
                    'email': email,
                    'password': hashed_password  # Store hashed password
                }
            )

            return redirect(url_for('login'))

        except Exception as e:
            app.logger.error(f"Signup error: {str(e)}")
```

Description: Collecting registration data, and stores user details in the database.

- **Login Route (GET/POST):**

```python
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        try:
            # Fetch user from DynamoDB
            response = users_table.get_item(Key={'username': username})

            if 'Item' not in response:
                return render_template('login.html', error='User not found')

            user = response['Item']

            # Ensure password field exists in the DB
            if 'password' not in user:
                return render_template('login.html', error='Password not found in database')

            # Verify password
            if check_password_hash(user['password'], password):
                session['logged_in'] = True
                session['username'] = username
                session.setdefault('cart', [])  # Initialize cart if not set
```

Description: Verifies user credentials, increments login count, and redirects to the dashboard on success.

- **Home Route:**

```python
@app.route('/home')
def home():
    if not session.get('logged_in'):
        return redirect(url_for('login'))
    return render_template('home.html')

@app.route('/non_veg_pickles')
def non_veg_pickles():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    return render_template('non_veg_pickles.html', products=products['non_veg_pickles'])

@app.route('/veg_pickles')
def veg_pickles():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    # Simply pass all products without filtering
    return render_template('veg_pickles.html', products=products['veg_pickles'])

@app.route('/snacks')
def snacks():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    return render_template('snacks.html', products=products['snacks']
```

Restart Visual Studio

Description: Home page contains the routing for different categories which are Veg_pickles, Non_Veg_pickles, Snacks.

- **Check out Route:**

```python
@app.route('/checkout', methods=['GET', 'POST'])
def checkout():
    if not session.get('logged_in'):
        return redirect(url_for('login'))

    error_message = None  # Variable to hold error messages

    if request.method == 'POST':
        try:
            # Extract form data safely
            name = request.form.get('name', '').strip()
            address = request.form.get('address', '').strip()
            phone = request.form.get('phone', '').strip()
            payment_method = request.form.get('payment', '').strip()

            # Validate inputs
            if not all([name, address, phone, payment_method]):
                return render_template('checkout.html', error="All fields are required.")

            if not phone.isdigit() or len(phone) != 10:
                return render_template('checkout.html', error="Phone number must be exactly 10 digits.")

            # Get cart data from hidden inputs
            cart_data = request.form.get('cart_data', '[]')
            total_amount = request.form.get('total_amount', '0')
```

Restart Visual Studio Code to apply the late

```
            # Redirect to success page with success message
            return redirect(url_for('sucess', message="Your order has been placed successfully!"))

        except Exception as e:
            print(f"Checkout error: {str(e)}")
            return render_template('checkout.html', error="An unexpected error occurred. Please try again.")

    return render_template('checkout.html')  # Render checkout page for GET request


@app.route('/sucess')
def success():
    return render_template('sucess.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)  # Add debug=True temporarily
```

Description: Implement the checkout route to process orders by collecting user details from checkout.html, generating a unique order ID using UUID, and saving the transaction details to AWS DynamoDB via Boto3. After the data is securely stored, the route clears the user's session cart and redirects them to the success page for confirmation.

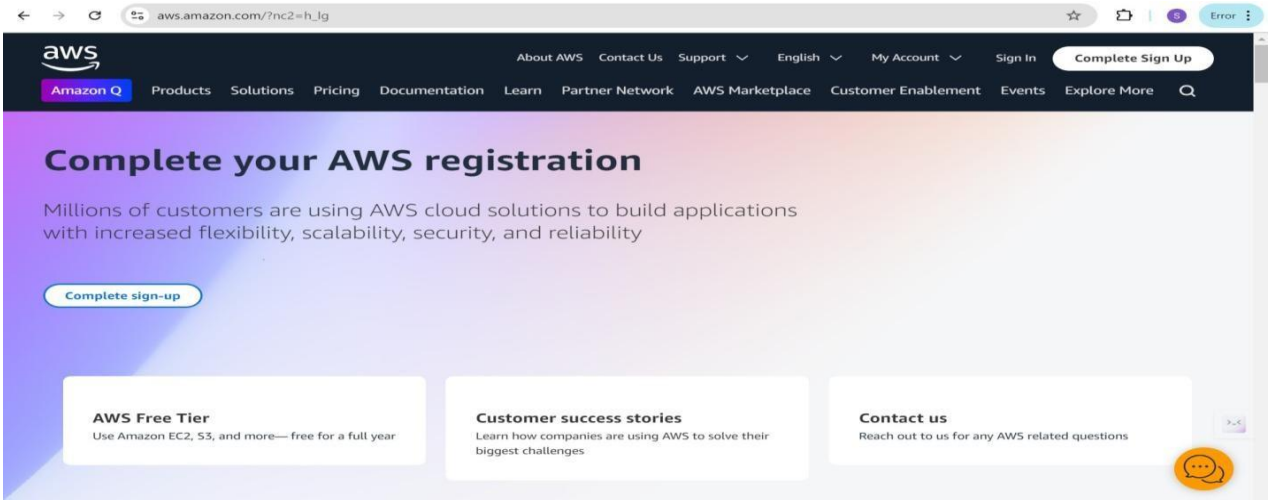
- **Logout route:**

```
@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))
```

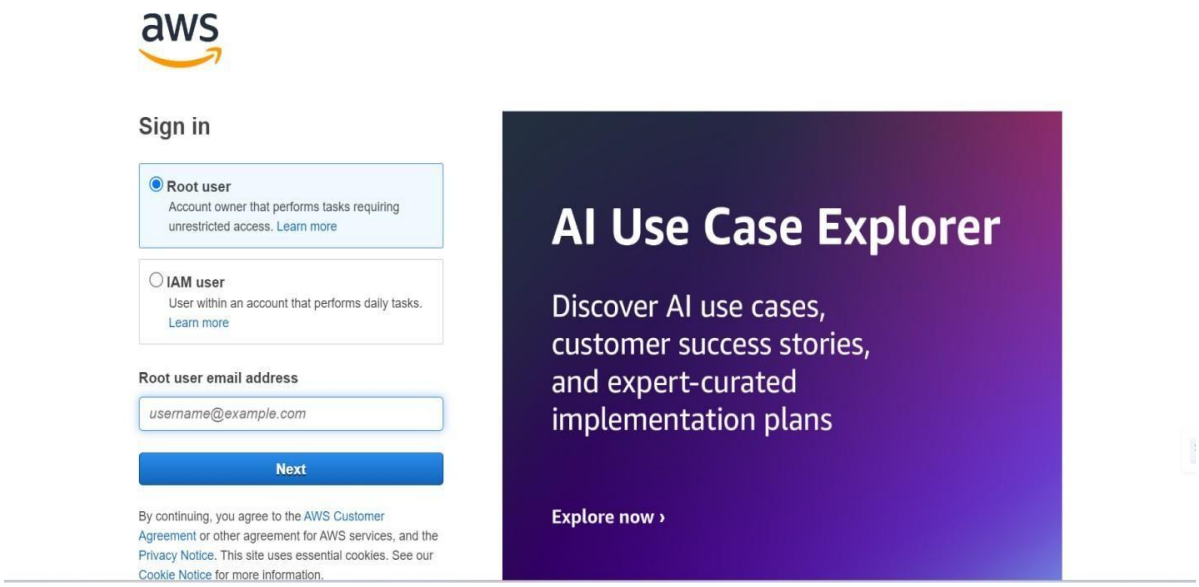
Description: The user can Logout so that the user can get back to the Login Page

Milestone 2: AWS Account Setup and Login

- **Activity 1.1: Set up an AWS account if not already done.**
    - Sign up for an AWS account and configure billing settings.

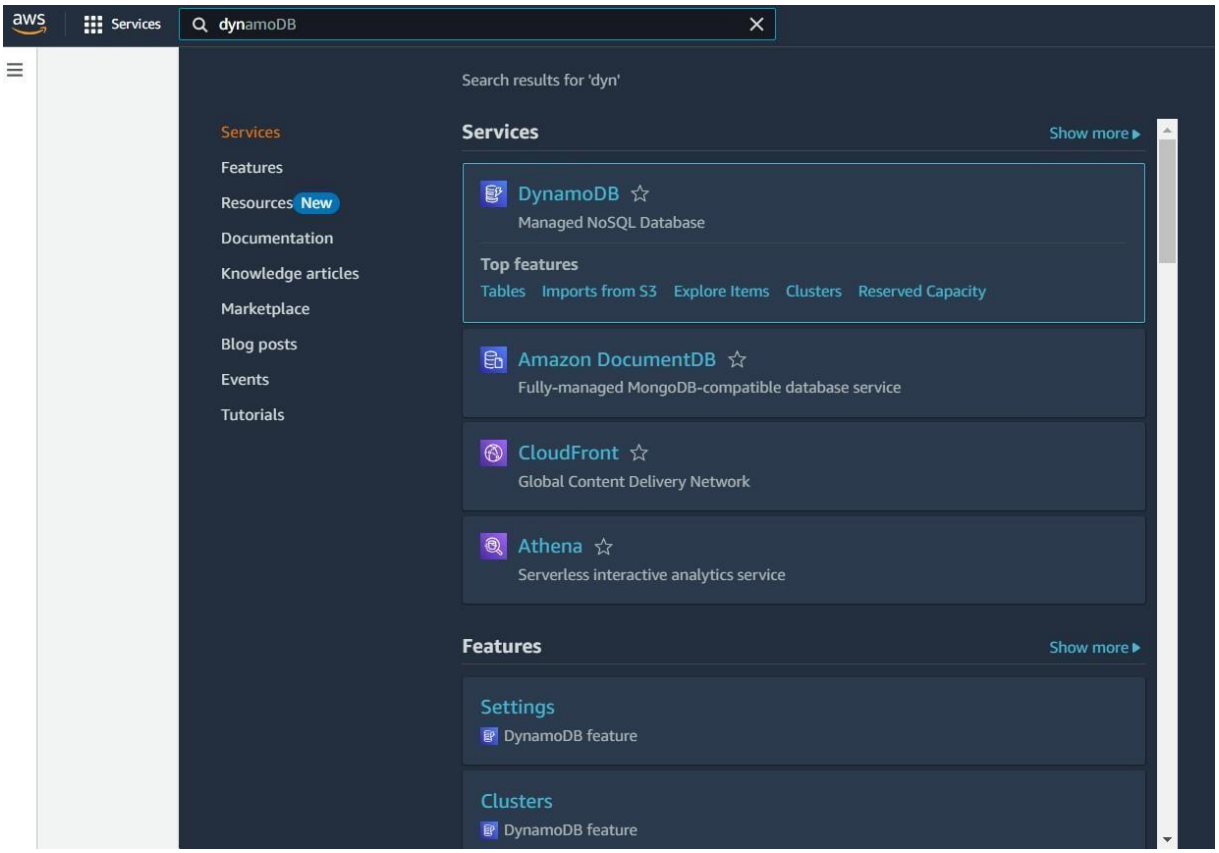


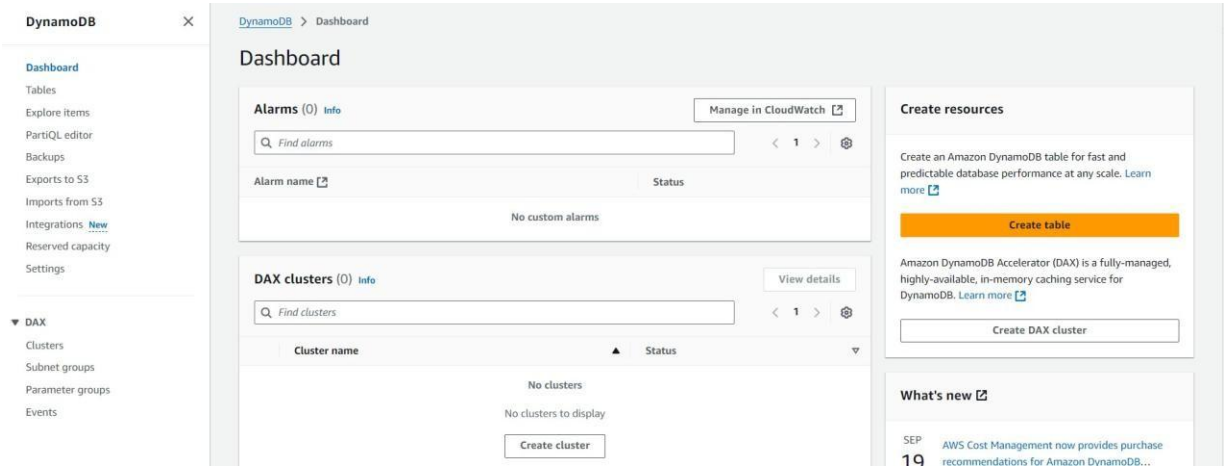
- **Activity 1.2: Log in to the AWS Management Console**
    - After setting up your account, log in to the AWS Management Console.

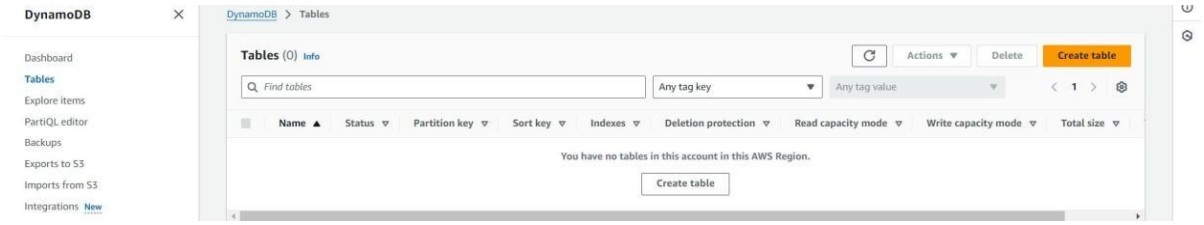


Milestone 3: DynamoDB Database Creation and Setup

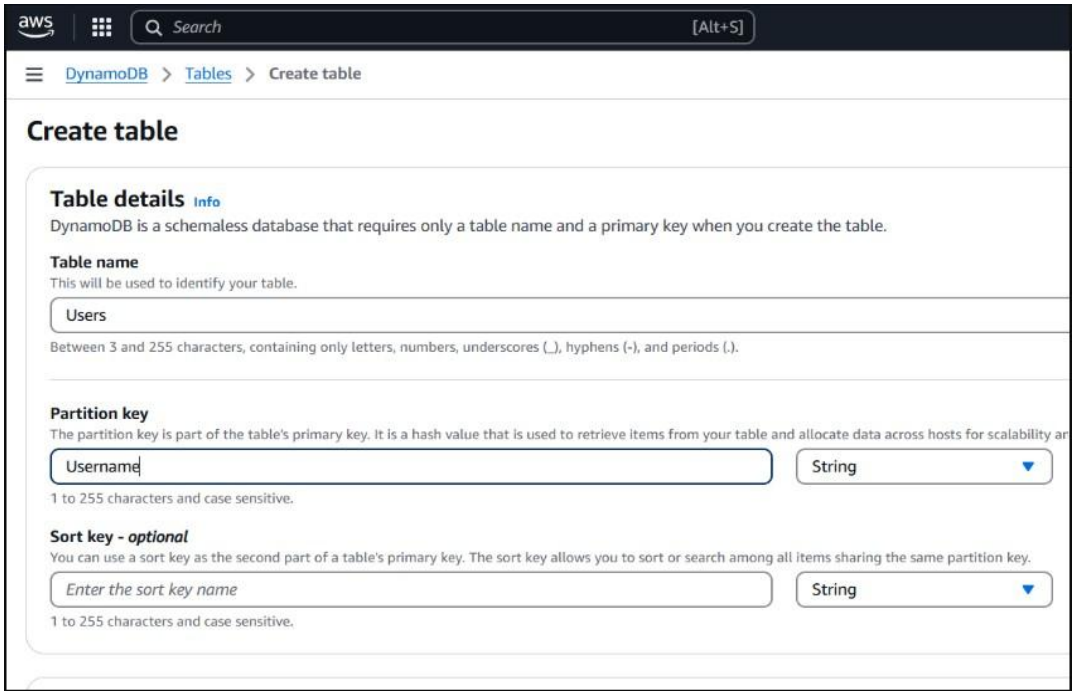
- **Activity 3.1: Navigate to the DynamoDB**
  - In the AWS Console, navigate to DynamoDB and click on create tables.

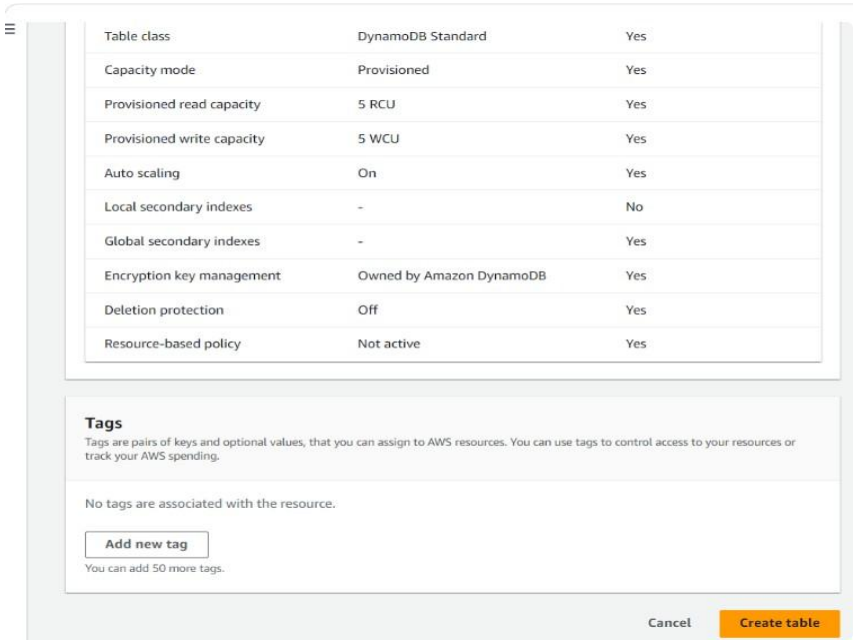


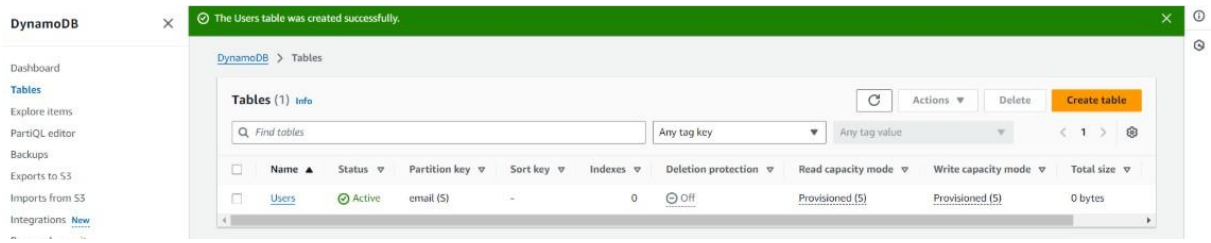




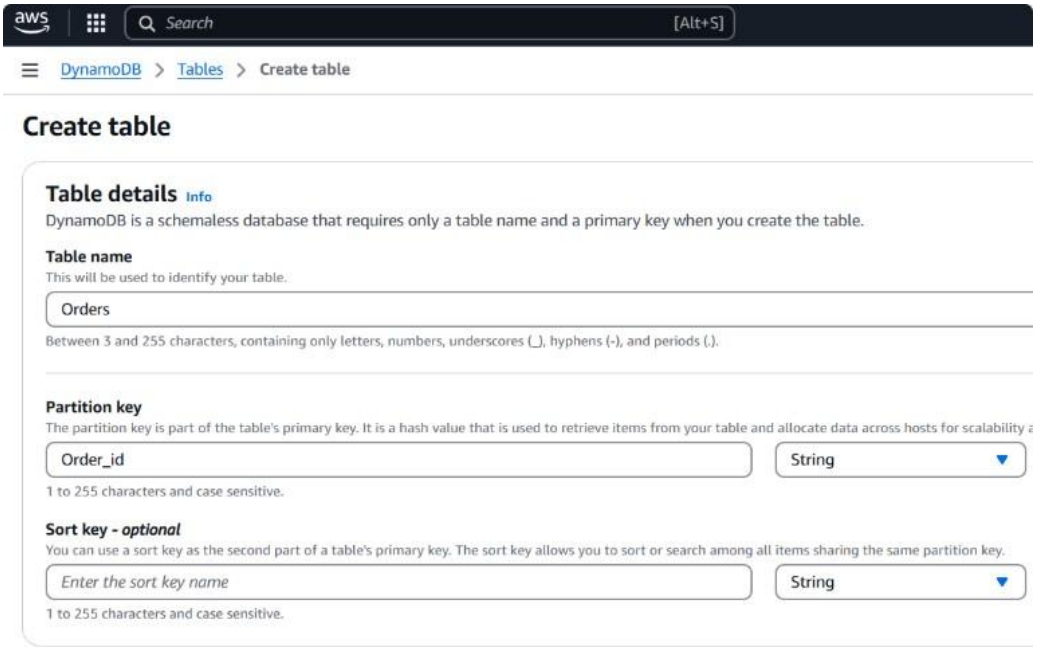
- **Activity 3.2 Create a DynamoDB table for storing data**
  - Create Users table with partition key "Username" with type String and click on create tables.





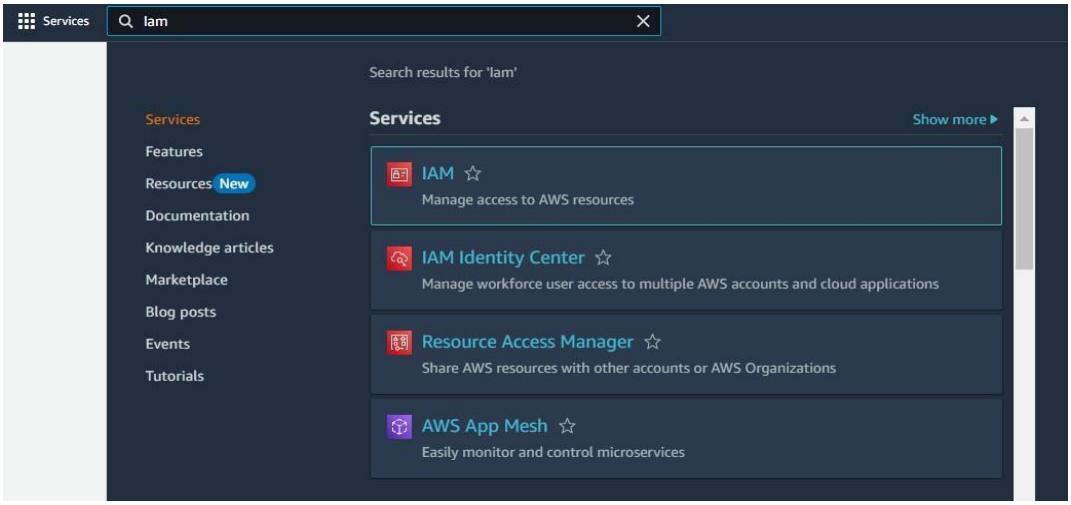


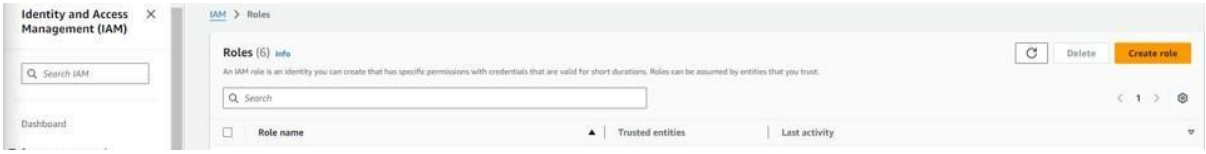
- Follow the same steps to create an Orders table with Order_id as the primary key to store Order details.

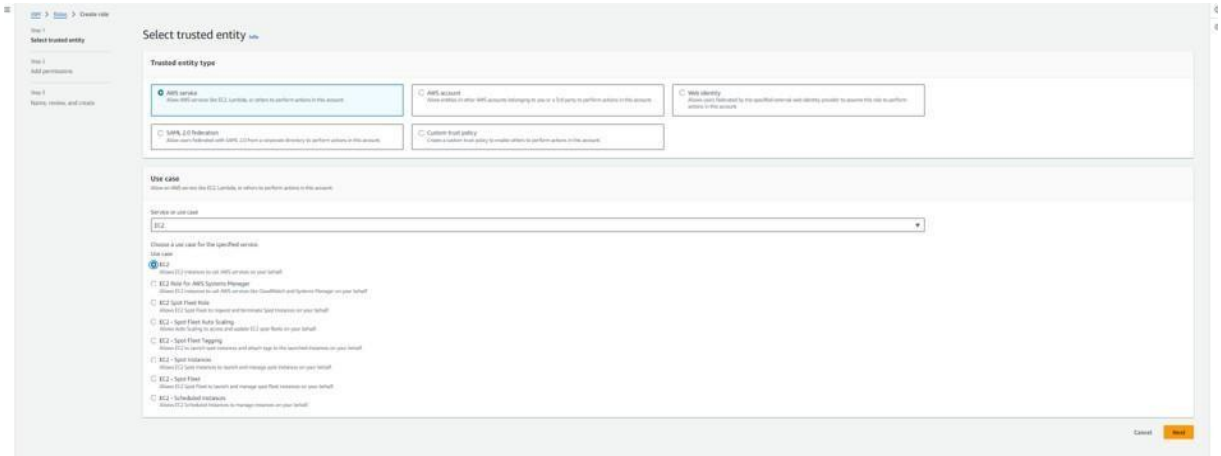


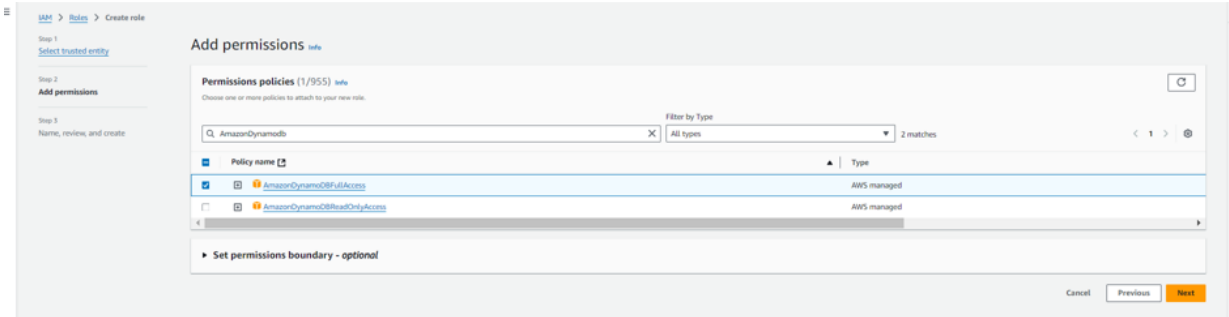
Milestone 4: IAM Role Setup

- **Activity 4.1: Create IAM Role.**
  - In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB





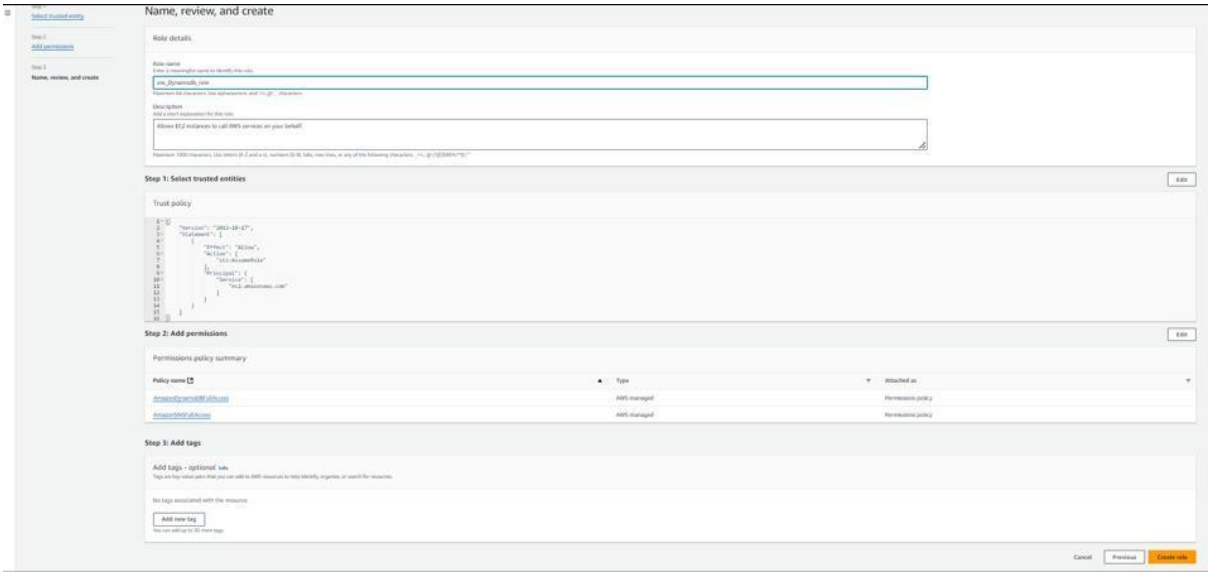


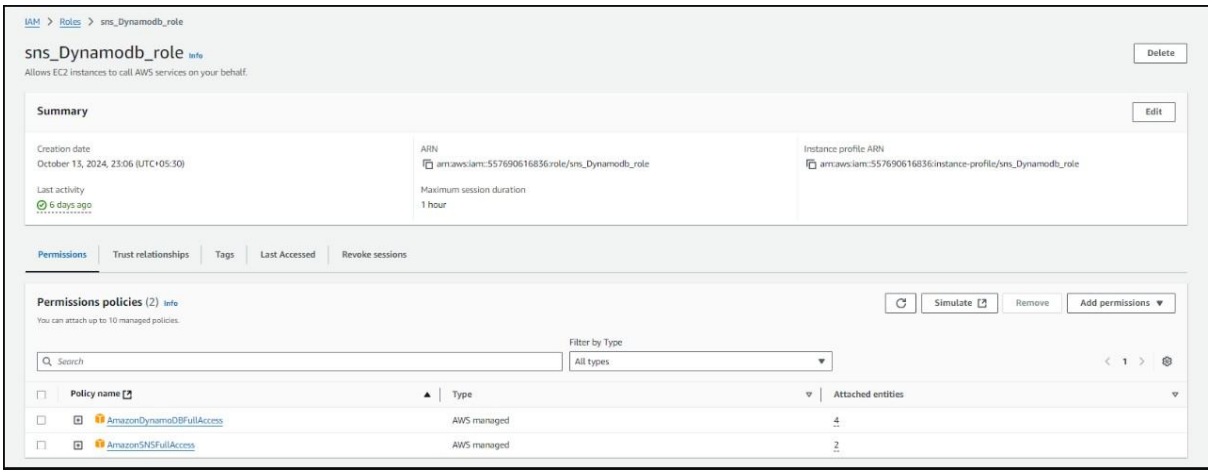


- **Activity 4.2: Attach Policies.**

  Attach the following policies to the role:

  AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.

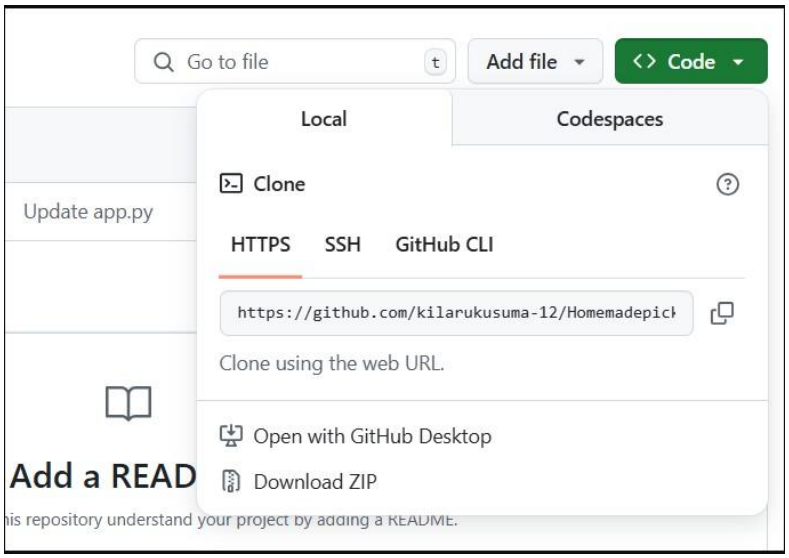




Milestone 5: EC2 Instance Setup

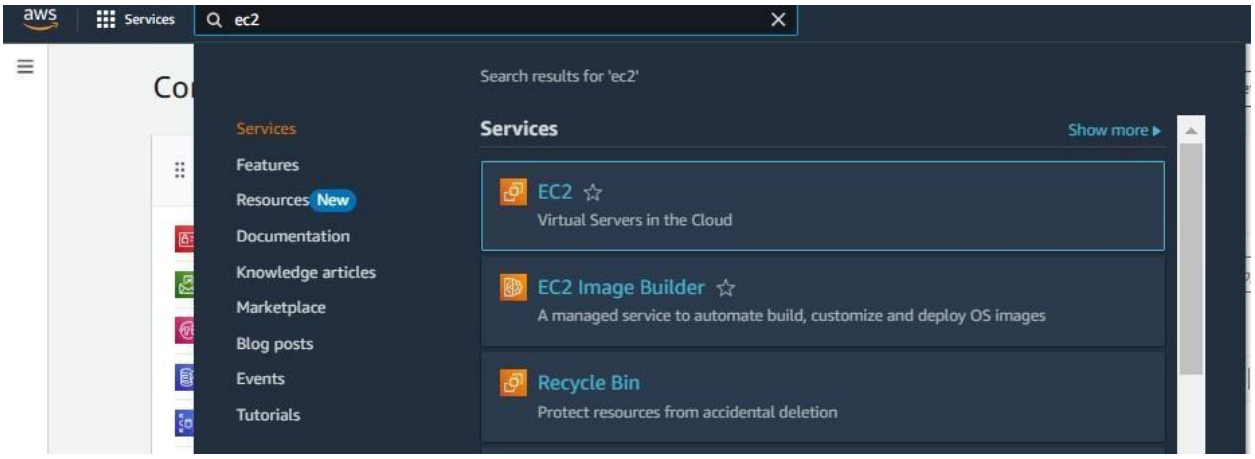
- **Note: Load your Flask app and Html files into GitHub repository.**

static	Initial commit
templates	Update statistics.html
app.py	Update app.py

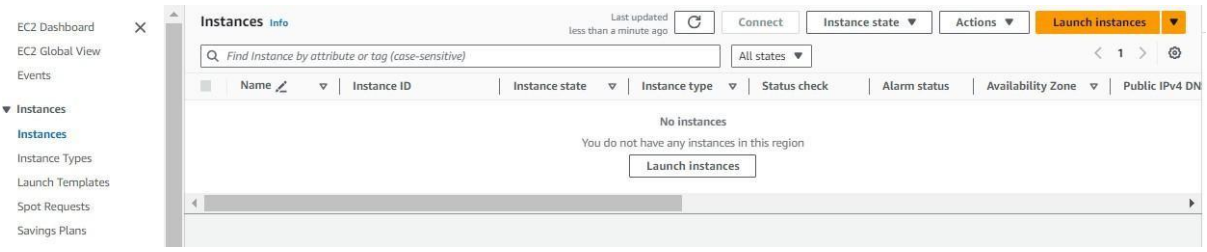


Activity 5.1: Launch an EC2 instance to host the Flask application.

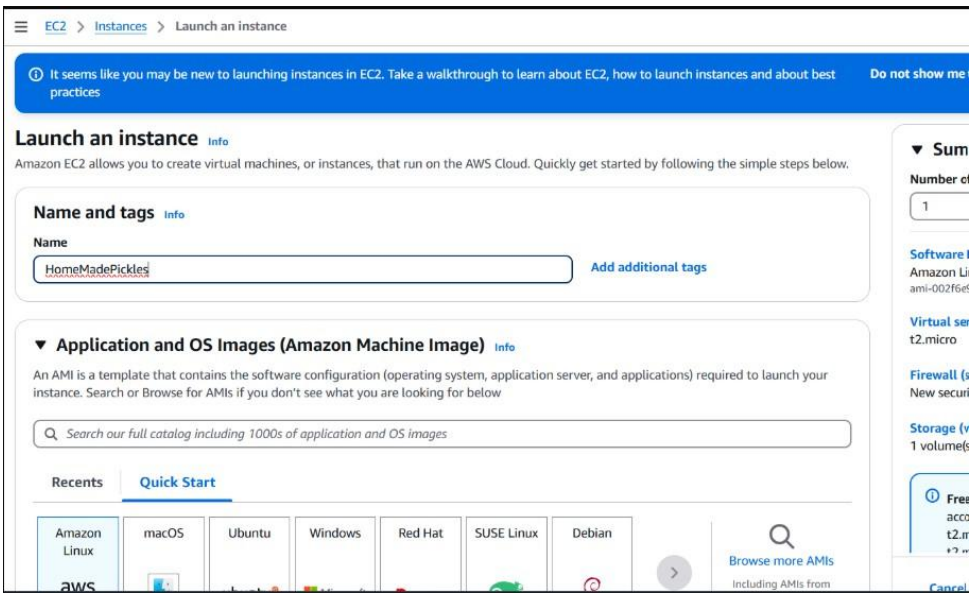
- **Launch EC2 Instance**
  - In the AWS Console, navigate to EC2 and launch a new instance.



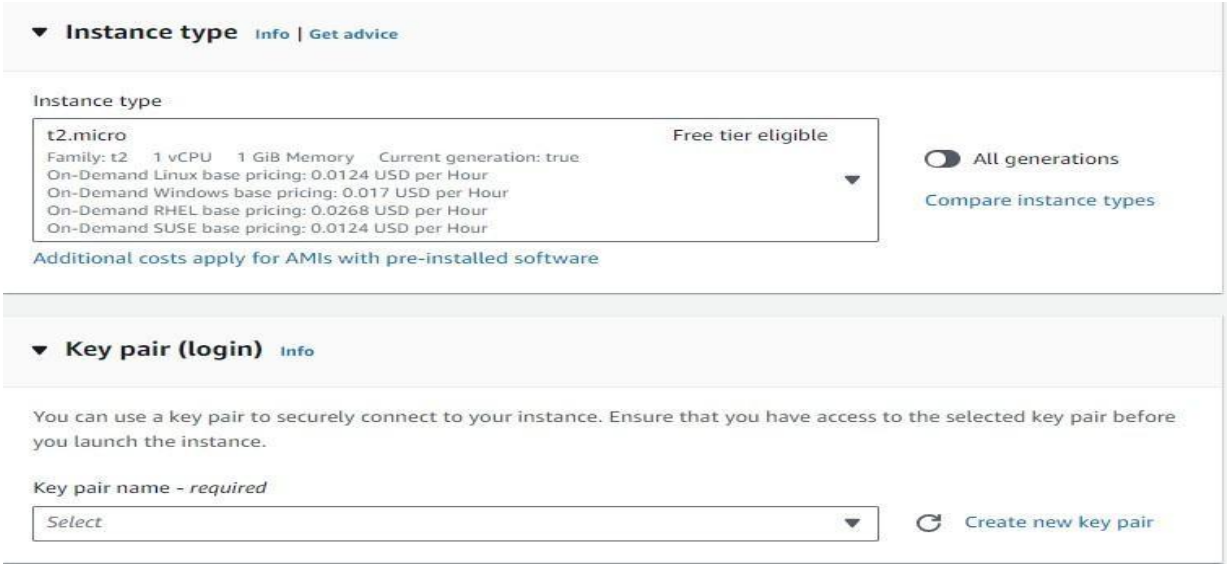
  - Click on Launch instance to launch EC2 instance

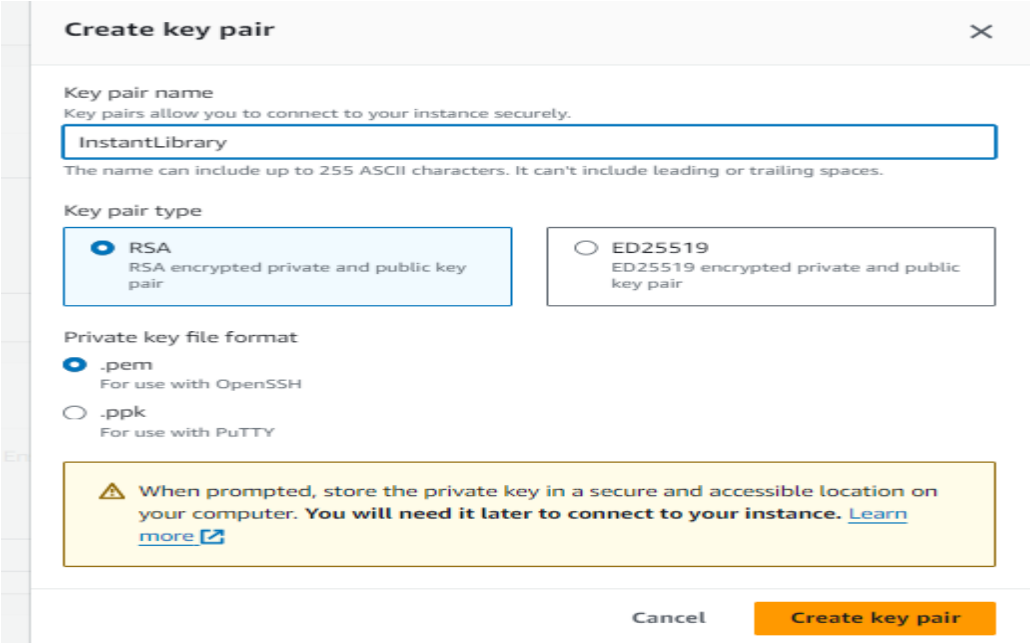


- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instance type (free-tier eligible).

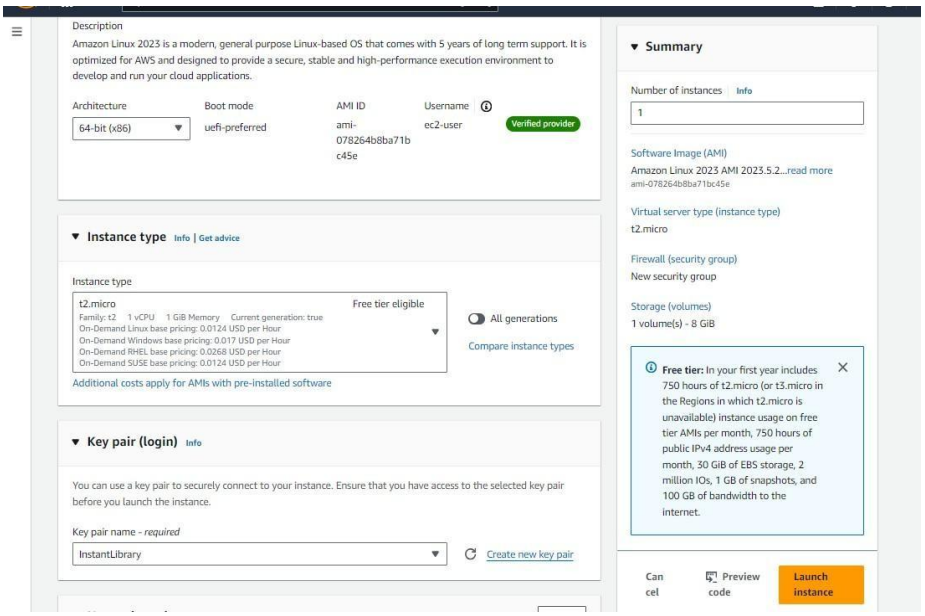


- Create and download the key pair for Server access.

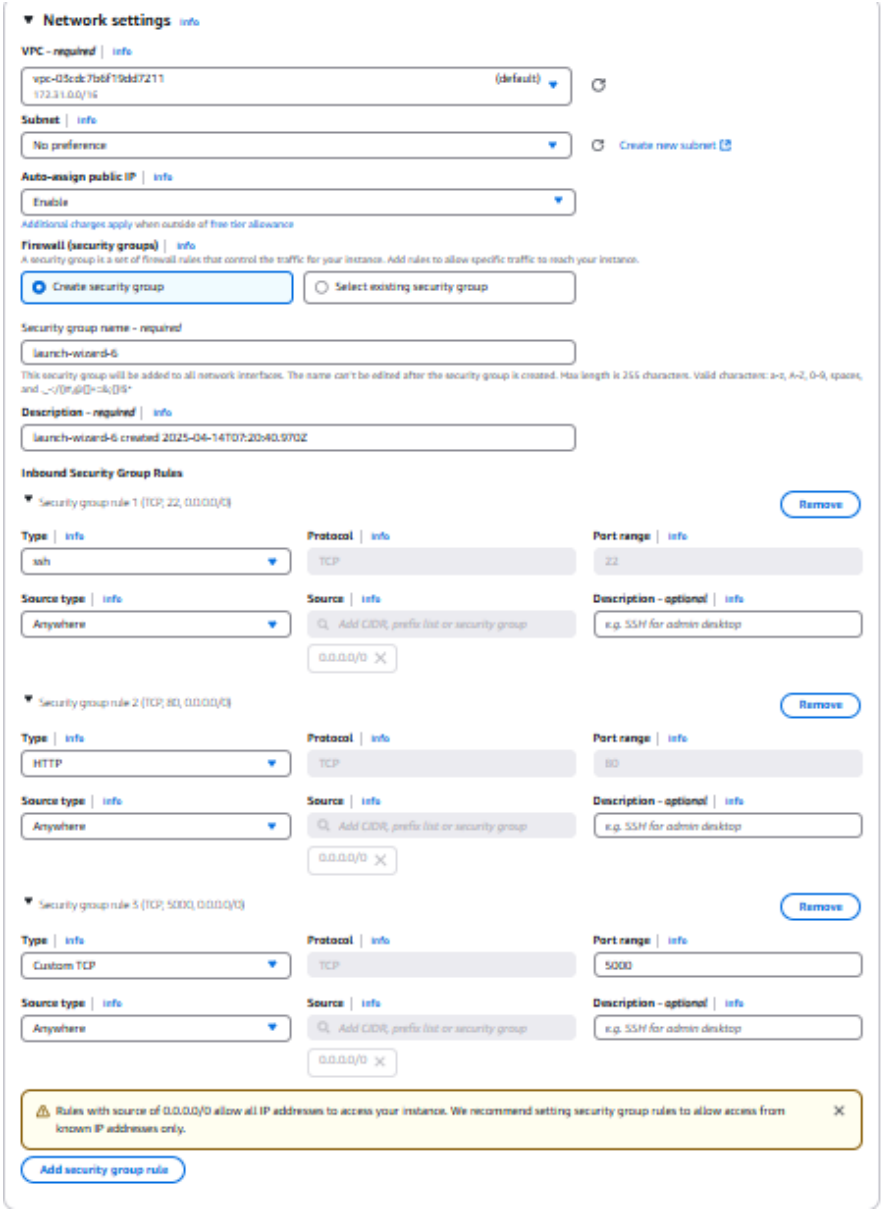


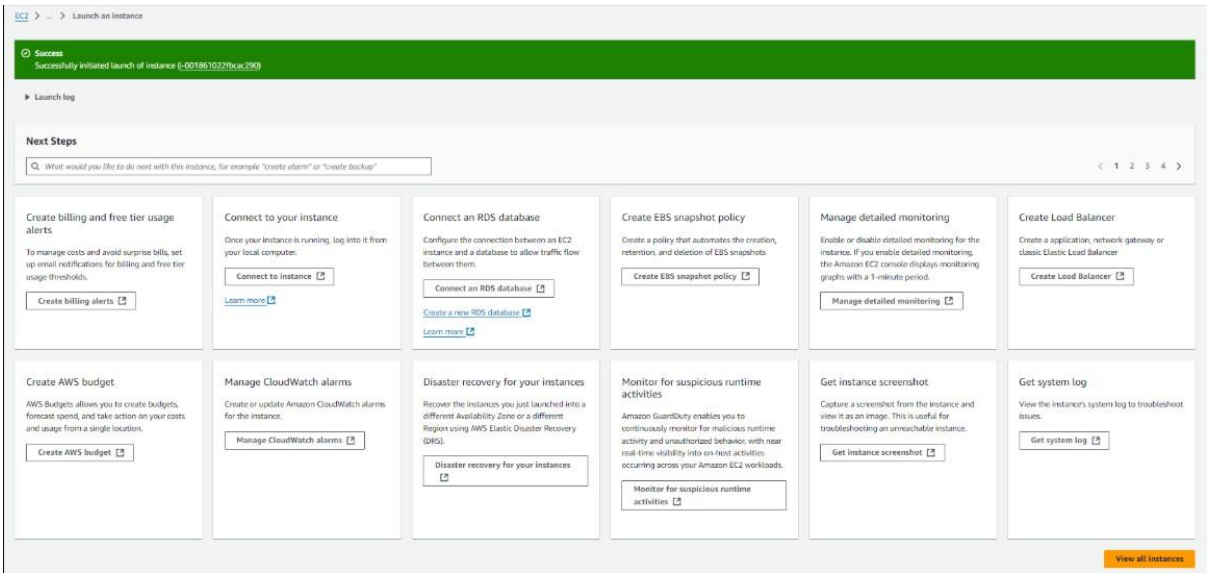


InstantLibrary.pem

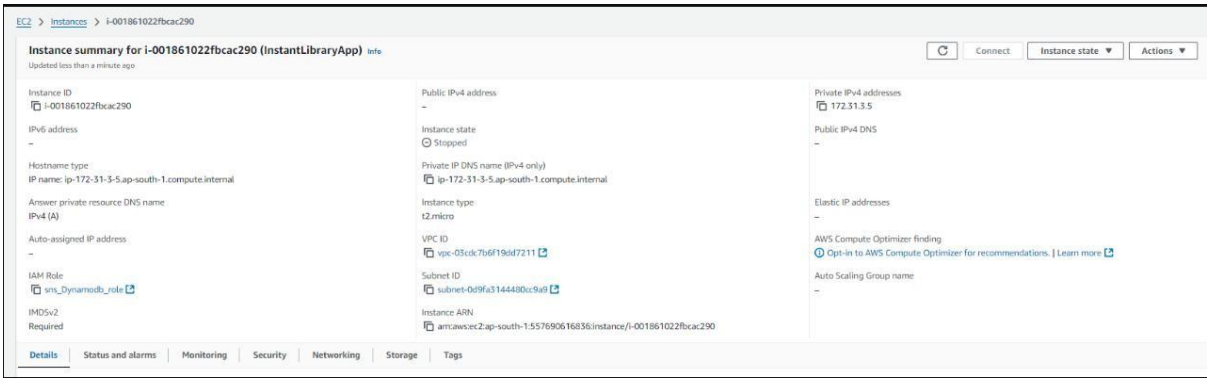


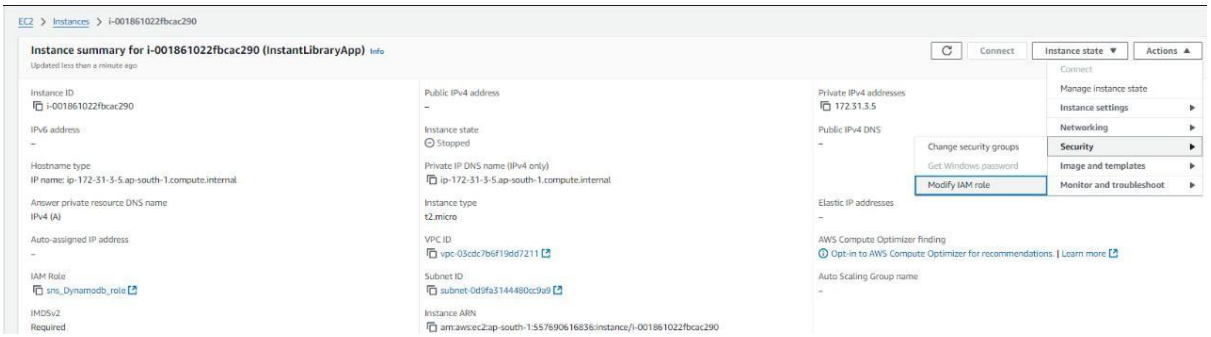
- **Activity 5.2: Configure security groups for HTTP, and SSH access.**

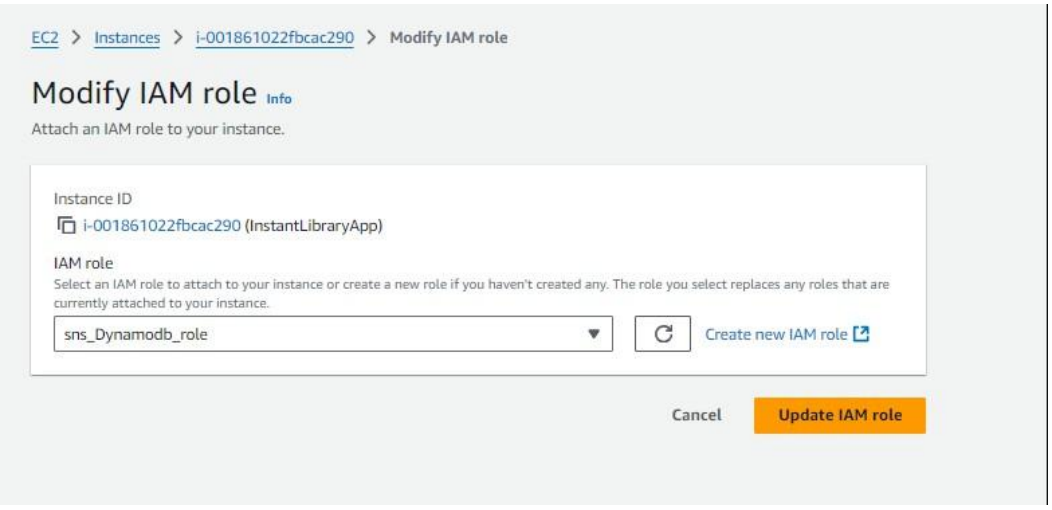




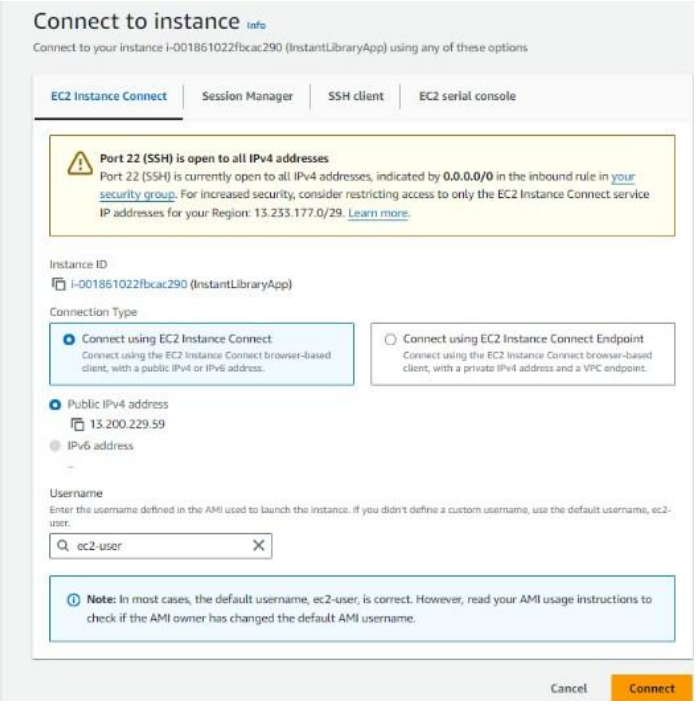
To connect to EC2 using **EC2 Instance Connect**, start by ensuring that an **IAM role** is attached to your EC2 instance. You can do this by selecting your instance, clicking on **Actions**, then navigating to **Security** and selecting **Modify IAM Role** to attach the appropriate role. After the IAM role is connected, navigate to the **EC2** section in the **AWS Management Console**. Select the **EC2 instance** you wish to connect to. At the top of the **EC2 Dashboard**, click the **Connect** button. From the connection methods presented, choose **EC2 Instance Connect**. Finally, click **Connect** again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.







- Now connect the EC2 with the files



Milestone 6: Deployment on EC2

Activity 6.1: Install Software on the EC2 Instance

Install Python3, Flask, and Git

On Amazon Linux 2:

sudo yum update -y

sudo yum install python3 git

sudo pip3 install flask boto3

Verify Installations:

flask --version

git –version

Clone your project repository from GitHub into the EC2 instance using Git.

Run: 'git clone https://github.com/your-github-username/your-repository-name.git'

Note: change your-github-username and your-repository-name with your credentials here: 'https://github.com/Manya-DA/Homemade-pickles-snacks-AWS_cloud_project'

This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

cd Homemadepicklesandsnacks

cd “Home Made Pickles1”

Create a Virtual Environment:

python3 -m venv venv

source venv/bin/activate

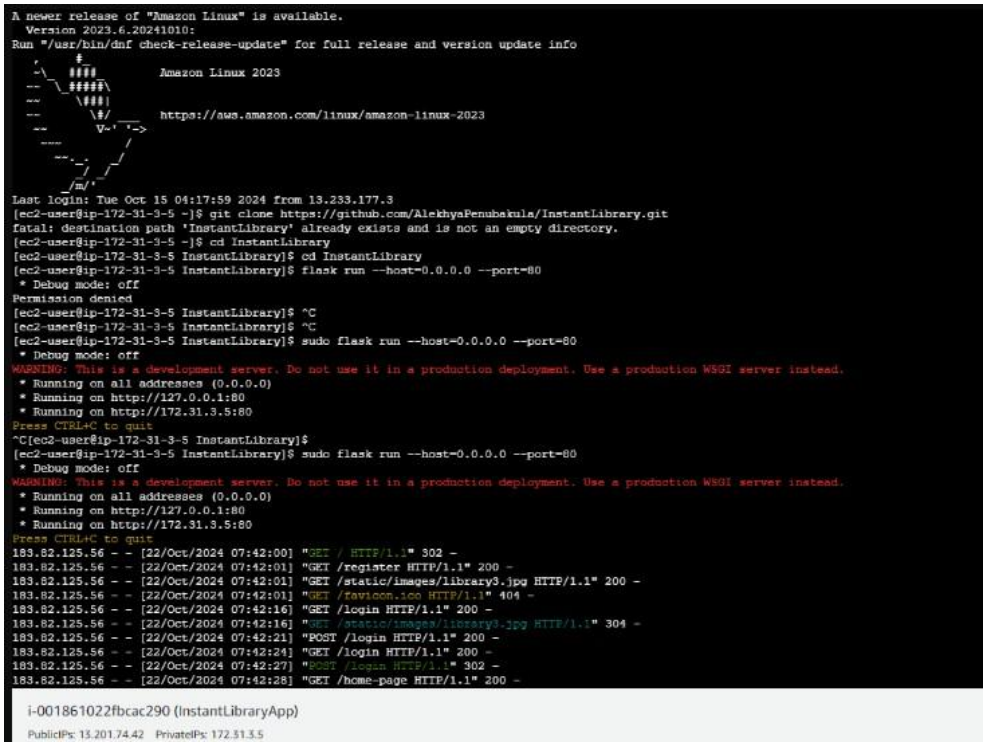
sudo yum install python3 git

sudo pip3 install flask boto3

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Activity 6.2: Run the Flask Application

sudo flask run --host=0.0.0.0 --port=5000



Verify the Flask app is running:

http://your-ec2-public-ip

Run the Flask app on the EC2 instance

```
ModuleNotFoundError: No module named 'boto3'

[ec2-user@ip-172-31-15-149 Home made pickles1]$ sudo pip3 install boto3
Collecting boto3
  Downloading boto3-1.37.23-py3-none-any.whl (139 kB)
     |████████████████████████████████| 139 kB 8.1 MB/s
Collecting s3transfer<0.12.0,>=0.11.0
  Downloading s3transfer-0.11.4-py3-none-any.whl (84 kB)
     |████████████████████████████████| 84 kB 6.7 MB/s
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /usr/lib/python3.9/site-packages (from boto3) (0.10.0)
Collecting botocore<1.38.0,>=1.37.23
  Downloading botocore-1.37.23-py3-none-any.whl (13.4 MB)
     |████████████████████████████████| 13.4 MB 38.3 MB/s
Requirement already satisfied: urllib3<1.27,>=1.25.4 in /usr/lib/python3.9/site-packages (from botocore<1.38.0,>=1.37.23->boto3) (1.25.10)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /usr/lib/python3.9/site-packages (from botocore<1.38.0,>=1.37.23->boto3) (2.8.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil<3.0.0,>=2.1->botocore<1.38.0,>=1.37.23->boto3) (1.15.0)
Installing collected packages: botocore, s3transfer, boto3
Successfully installed boto3-1.37.23 botocore-1.37.23 s3transfer-0.11.4
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual
environment instead: https://pip.pypa.io/warnings/venv
[ec2-user@ip-172-31-15-149 Home made pickles1]$ sudo flask run --host=0.0.0.0 --port=5000
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.31.15.149:5000
Press CTRL+C to quit
```

Access the website through:

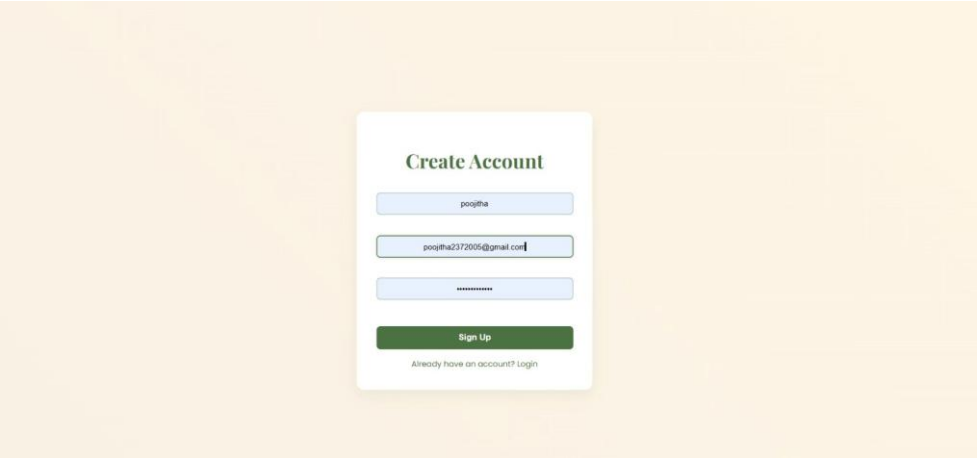
Public IP: http://13.233.104.34:5000

Milestone 7: Testing and Deployment

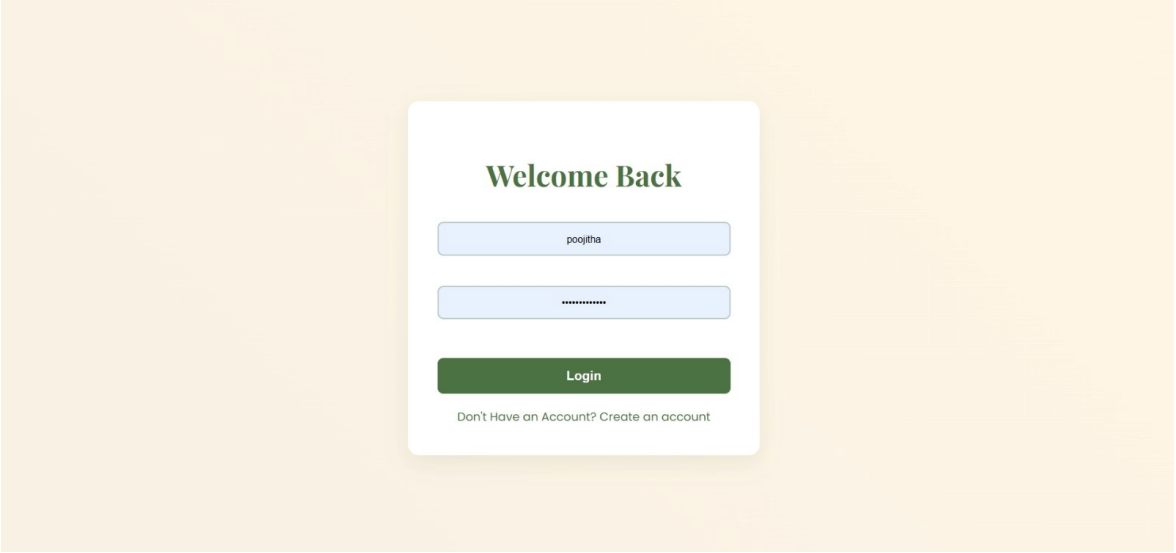
- **Activity 7.1: Conduct functional testing to verify the Project**

Welcome page:

Signup page:

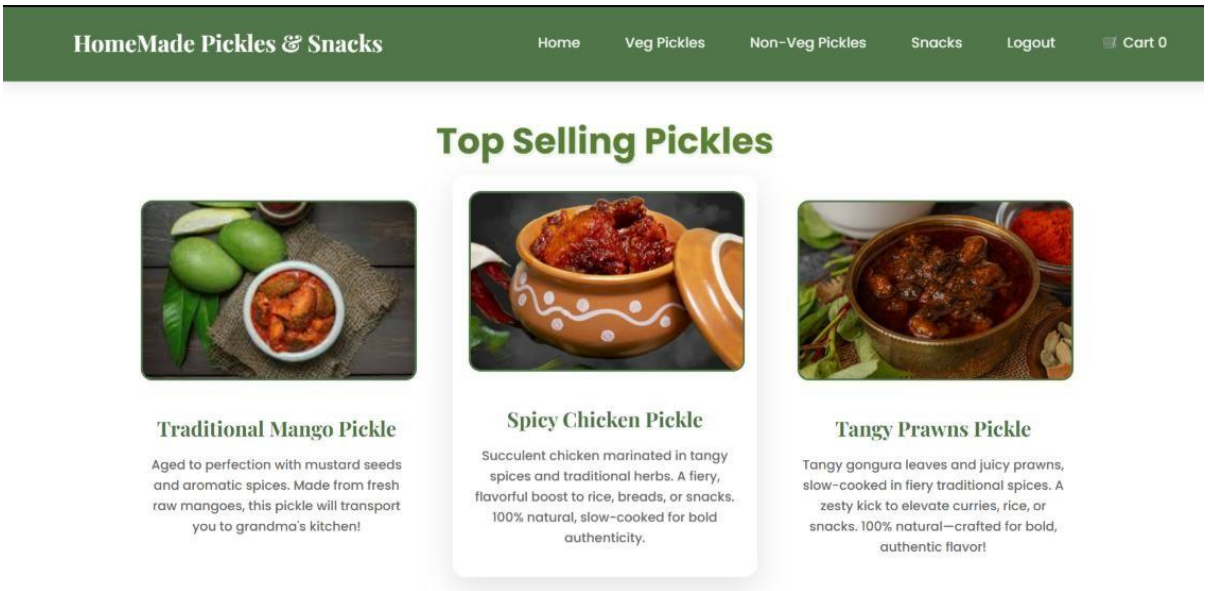


Login Page:



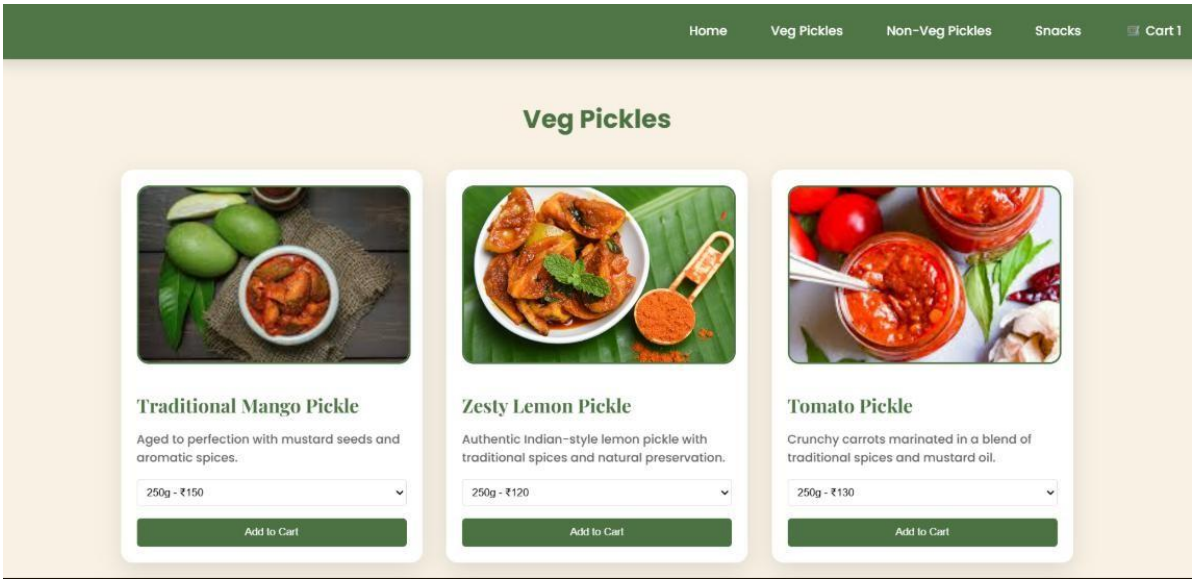
Home page:

HomeMade Pickles & Snacks
Home
Veg Pickles
Non-Veg Pickles
Snacks
Logout
Cart 0
Welcome to HomeMade Pickles & Snacks!
Your one-stop shop for delicious homemade pickles and snacks!

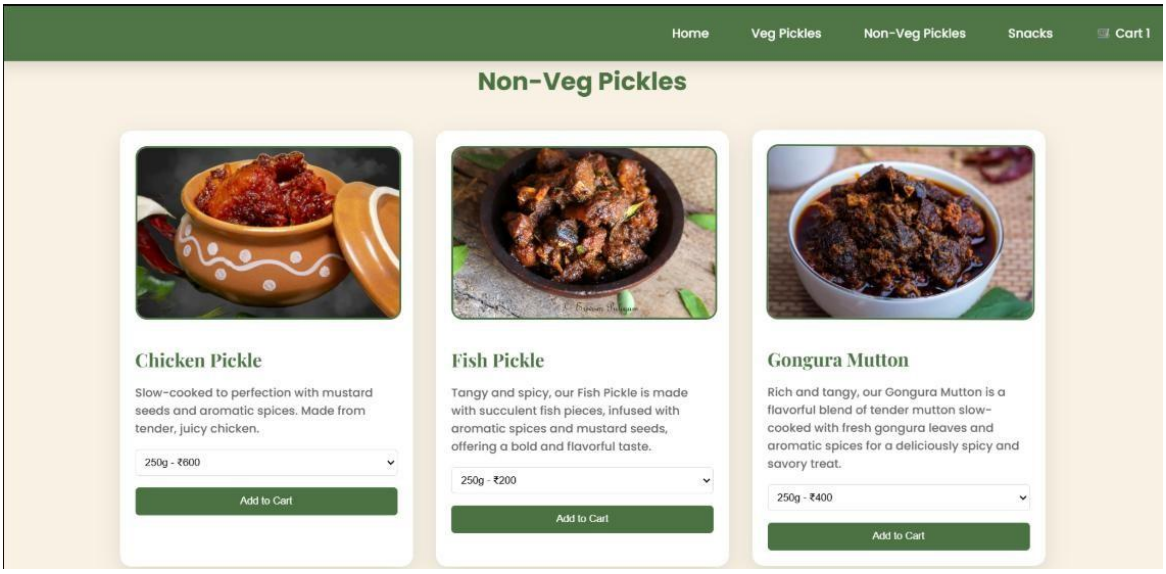

HomeMade Pickles & Snacks
Home
Veg Pickles
Non-Veg Pickles
Snacks
Logout
Cart 0
Top Selling Pickles
Traditional Mango Pickle
Aged to perfection with mustard seeds and aromatic spices. Made from fresh raw mangoes, this pickle will transport you to grandma's kitchen!
Spicy Chicken Pickle
Succulent chicken marinated in tangy spices and traditional herbs. A fiery, flavorful boost to rice, breads, or snacks. 100% natural, slow-cooked for bold authenticity.
Tangy Prawns Pickle
Tangy gongura leaves and juicy prawns, slow-cooked in fiery traditional spices. A zesty kick to elevate curries, rice, or snacks. 100% natural—crafted for bold, authentic flavor!

HomeMade Pickles & Snacks
Home
Veg Pickles
Non-Veg Pickles
Snacks
Logout
Cart 0
Summer Special Snacks
Mango Aam Papad
Sweet & tangy mango leather, sun-dried to perfection. Kids' favorite summer treat with no added preservatives.
Crispy Banana Chips
Thinly sliced raw bananas sun-dried and lightly salted. Gluten-free, vegan crunchy snack for all ages.
Sunnundalu
Sunnundalu, a traditional sweet ball popular in Andhra Pradesh, are made primarily from urad dal (black lentils), jaggery or sugar, ghee, and cardamom

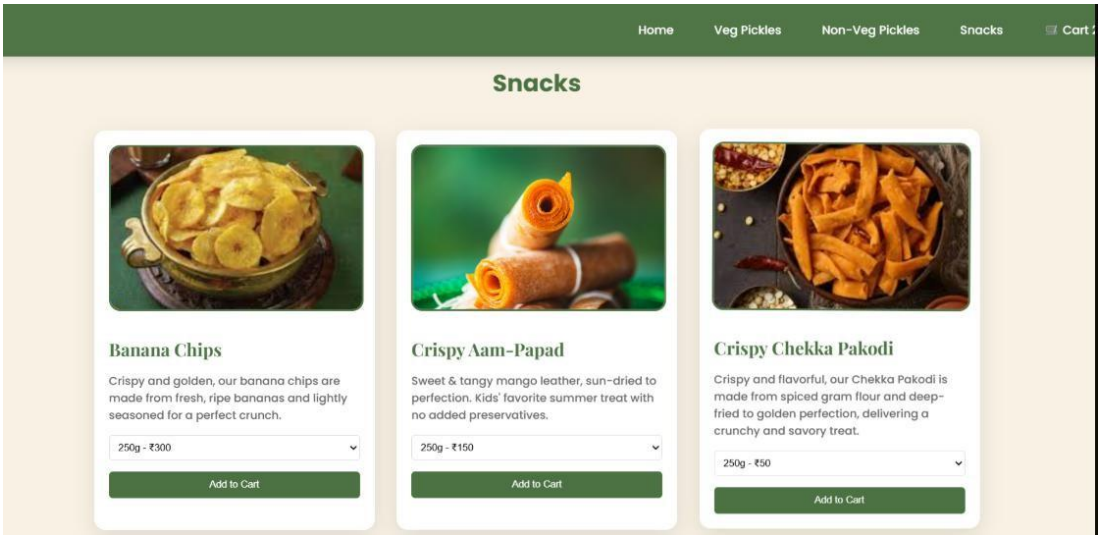
Veg Pickles Page:



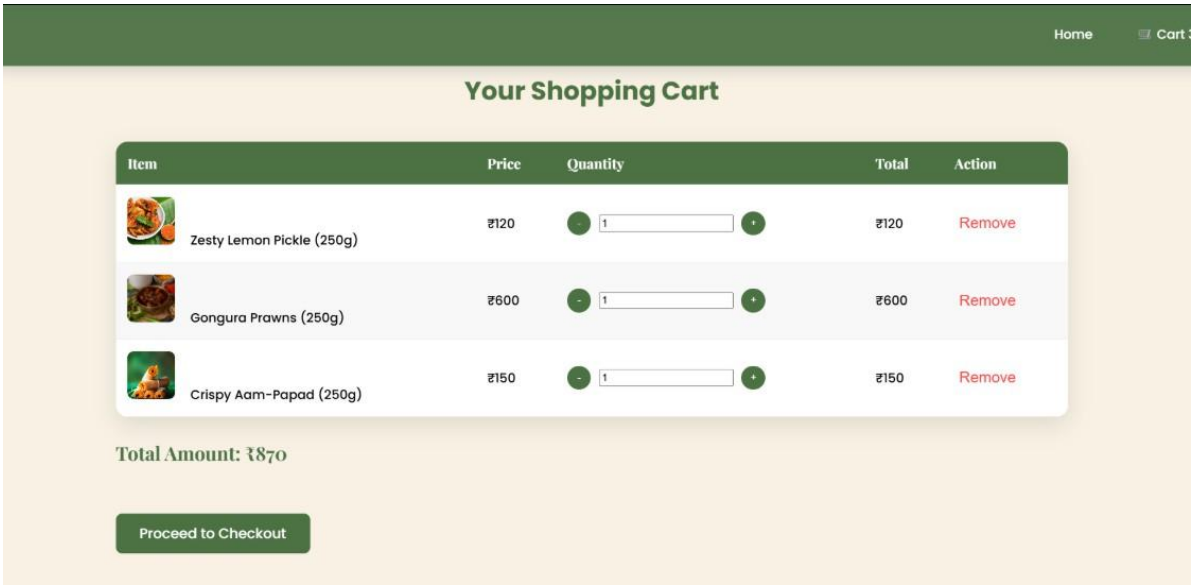
Non-Veg Pickles page:



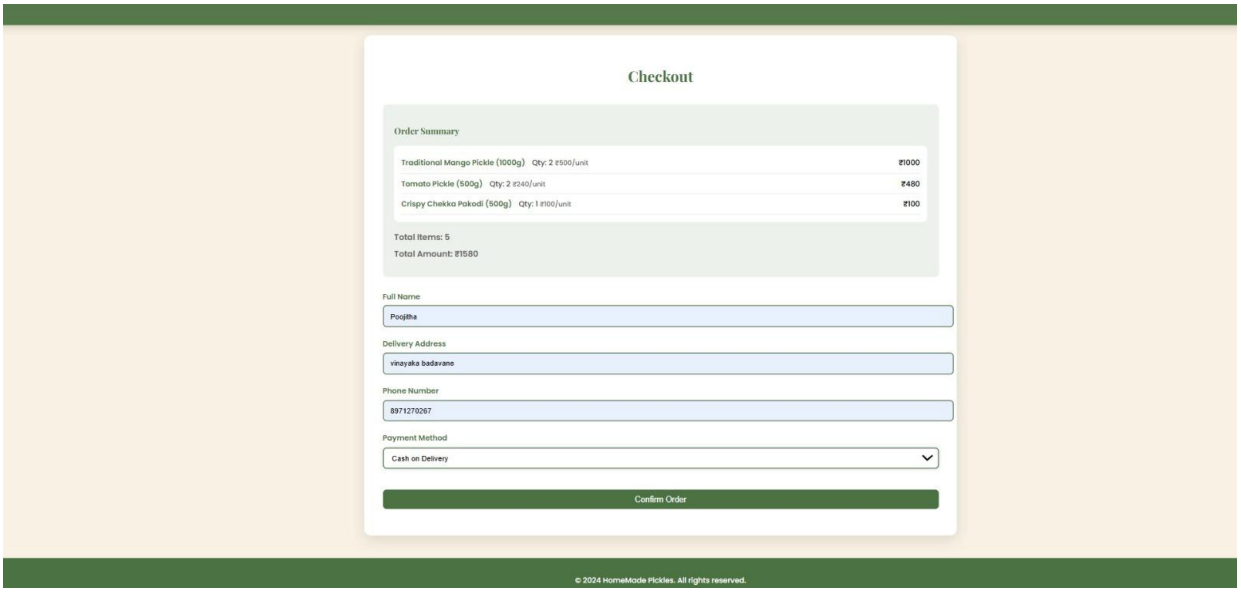
Snacks Page:



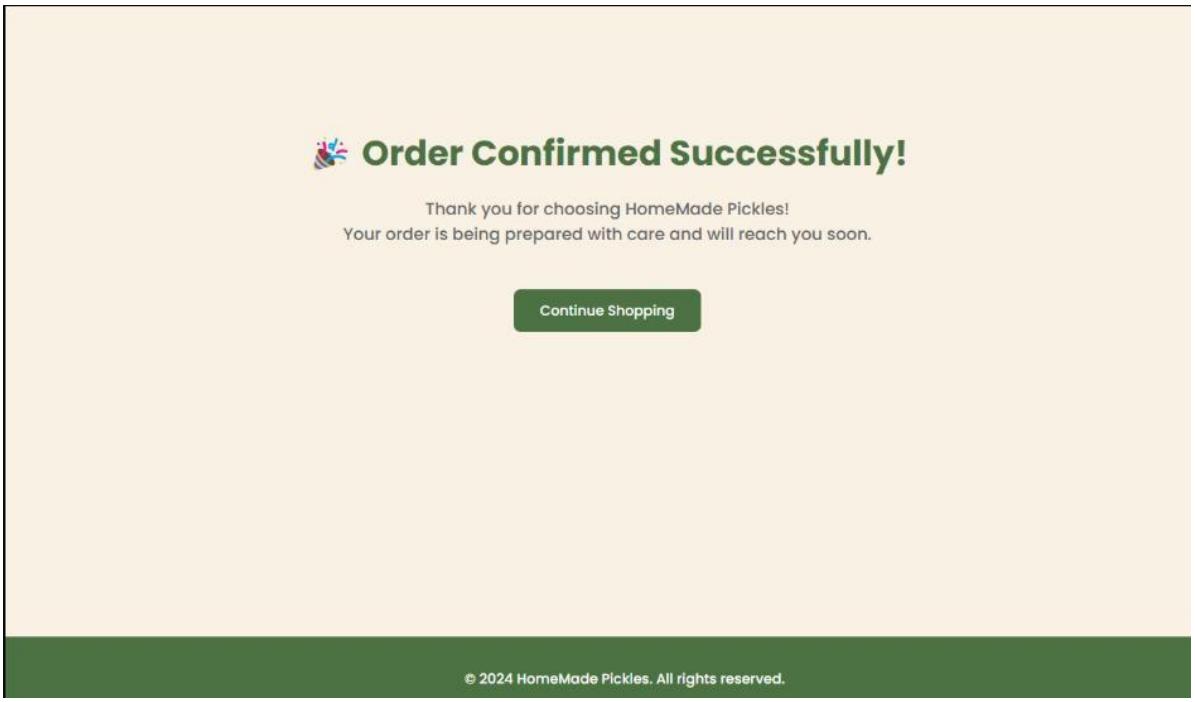
Cart Page:



Check out Page:



Success page:



DynamoDB Database updates:

1. Users table :

Completed. Read capacity units consumed: 2

Items returned (3)

Actions ▼ Create item

1

username *(String)*	email	password
Shiva	kilarukusu...	scrypt:32768:8:1$W5tA59Z7nQjLXbtx$d6bfef2b3e14bbe9d3d3e3f1c...
kusuma	<empty>	<empty>
Alekhya	alekhya@g...	scrypt:32768:8:1$EwCDTI0iaGcKutw3$cd5dbf5c12ec17cb518f7c15cd...

2. Orders table :

Conclusion:

The Homemade Pickles and Snacks platform has been meticulously crafted to deliver a seamless and delightful experience for food enthusiasts seeking authentic, handcrafted flavours. By leveraging modern web technologies such as Flask for backend logic, secure user authentication, and dynamic cart management, the platform ensures a user-friendly interface for browsing, customizing, and ordering artisanal pickles and snacks.

The integration of cloud-ready architecture (e.g., AWS for future scalability) and robust session management allows the platform to handle high traffic efficiently while maintaining real-time updates for orders and inventory. Features like weight-based pricing, category-specific searches, and instant checkout streamline the shopping process, empowering customers to explore a diverse range of traditional and innovative recipes with ease.

This project addresses the growing demand for homemade, preservative-free food products by bridging the gap between small-scale producers and discerning customers. The platform's intuitive design and secure payment workflows enhance trust and convenience, while backend tools enable effortless inventory tracking and order fulfilment for administrators.

By combining time-honoured recipes with modern e-commerce capabilities, this website not only preserves culinary heritage but also adapts to the digital age, ensuring that every jar of pickle or snack reaches customers with the same care and quality as a homemade meal. As the platform evolves, it stands ready to scale, introduce new product lines, and foster a community of food lovers united by a passion for authentic flavours.

In essence, this project redefines the way homemade delicacies are shared and enjoyed, offering a flavourful bridge between tradition and technology.